DOCUMENTO DE HOMOLOGAÇÃO & ENGENHARIA DE SOFTWARE

PixFlow

Manual Oficial de Testes, Homologação e Auditoria Técnica

Guia técnico completo para validação funcional, testes de concorrência, auditoria de segurança criptográfica (HMAC-SHA256) e verificação de esteira CI/CD com 100% de cobertura de código.

Autor: Matheus Araújo Pereira — Cloud DevOps Engineer

Repositório Oficial: github.com/matheus-araujo-pereira/pix-flow

Ambientes Homologados: Vercel (Frontend SPA) • Render (Core Backend Java 21) • Neon (PostgreSQL Serverless)

Versão do Motor: 1.0.0-PROD • **Licença:** MIT Open Source • **Custo Operacional:** 100% Gratuito (Free Tier)

1. Visão Geral da Arquitetura & Topologia de Testes

O **PixFlow** é um motor de mensageria e liquidação assíncrona de pagamentos instantâneos (padrão BACEN / SPI). Ele adota os princípios de *Clean Architecture (Arquitetura Hexagonal)* para garantir desacoplamento estrito entre a borda síncrona (recepção de ordens de pagamento via HTTP/JSON) e o processamento de liquidação bancária assíncrona gerenciado por workers de alta resiliência via **AWS SQS** (ou motor in-memory local).

URLs Públicas para Testes Online:

• **Interface Web Minimalista (Vercel):** https://frontend-hazel-six-05z2k7m6sk.vercel.app

• **Swagger / Documentação OpenAPI (Render):** https://pix-flow-backend.onrender.com/swagger-ui.html

• **Healthcheck Actuator:** `https://pix-flow-backend.onrender.com/actuator/health`

Diagrama de Ciclo de Estados da Transação

Estado	Descrição	Transições Permitidas
PENDING	Cobrança criada com EndToEndId e BR Code gerados; aguarda processamento do worker ou pagamento.	→ PROCESSING, CANCELED, SETTLED
PROCESSING	Worker retirou o evento da fila SQS; efetuando comunicação com clearing house e validações de saldo.	→ SETTLED, FAILED
SETTLED	Liquidação confirmada com sucesso; webhook assinado com HMAC-SHA256 emitido para o parceiro.	(Estado Final Imutável)
FAILED	Falha na liquidação bancária (ex: valor acima do threshold ou erro na câmara de compensação).	(Estado Final Imutável)
CANCELED	Cobrança cancelada a pedido do usuário antes do início da liquidação financeira.	(Estado Final Imutável)

2. Matriz de Massa de Dados & Validadores Canônicos (Módulo 11)

O motor PixFlow implementa no domínio matemático estrito a validação **Módulo 11** para identificadores de pessoas físicas (CPF) e jurídicas (CNPJ). Chaves com dígitos verificadores incorretos ou formadas por números repetidos (ex: `111.111.111-11`) são rejeitadas na borda com código **HTTP 422 Unprocessable Entity**.

Tabela de Chaves Canônicas Recomendadas para Testes

Tipo	Valor para Teste	Resultado Esperado	Notas de Homologação
CPF	`52998224725` (ou	VÁLIDO	Dígitos verificadores calculados via algoritmo

Tipo	Valor para Teste	Resultado Esperado	Notas de Homologação
	`529.982.247-25`)		oficial da Receita Federal.
CPF	`12345678911`	REJEITADO (422)	Dígito verificador inválido. Retorna erro canônico Módulo 11.
CPF	`11111111111`	REJEITADO (422)	Sequência de dígitos idênticos rejeitada preventivamente.
CNPJ	`04252011000110` (ou formatado)	VÁLIDO	CNPJ corporativo com pesos de 2 a 9 conferidos no Módulo 11.
CNPJ	`00000000000199`	REJEITADO (422)	Dígitos verificadores de controle incorretos.
EMAIL	`devops@pixflow.io`	VÁLIDO	Conforme padrão RFC 5322.
PHONE	`+5511999998888`	VÁLIDO	Padrão internacional ITU-T E.164 com código do país (+55).
RANDOM	`a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d`	VÁLIDO	Chave aleatória EVP padrão UUID v4 (36 caracteres alfanuméricos).

3. Guia Passo a Passo: Testes na Interface Web (SPA Vercel)

A interface do PixFlow foi desenvolvida com React, Vite e Tailwind CSS, focada em simplicidade operacional, alta densidade de dados e estética austera. A navegação sincroniza o estado via URLs com hash: `#terminal`, `#ledger`, `#stress`, `#webhooks` e `#tutorial`.

TELA 1: Terminal de Emissão PIX (#terminal) EMISSÃO SÍNCRONA

Objetivo: Emitir cobrança instantânea, validar máscara do formulário e gerar QR Code EMVCo.

1. Acesse a aba **"Terminal PIX"**.

2. Selecione o tipo de chave (ex: `CPF`) e clique em um dos atalhos de preenchimento rápido (ex: `529.982.247-25`).
3. Informe o valor (ex: `R$ 150,00`) e a descrição (ex: `Serviço Cloud DevOps`).
4. (Opcional) Insira uma URL de Webhook de retorno (ex: `https://webhook.site/teste`).
5. Clique no botão **"Emitir Cobrança PIX"**.
6. **Resultado Esperado:** Um card verde com o EndToEndId padrão BACEN é exibido, com o payload do Pix Copia e Cola (Tag 00 a Tag 63) e o botão para exibir o modal do QR Code.

TELA 2: Livro-Razão & Gaveta de Auditoria (#ledger) AUDITORIA IMUTÁVEL

Objetivo: Rastrear status em tempo real, abrir a timeline do worker e exportar dados em CSV.

1. Acesse a aba **"Livro-Razão"**.
2. Visualize a lista de transações ordenadas cronologicamente com filtros por status (PENDING, SETTLED, etc.).
3. Clique em qualquer linha para abrir a **Gaveta Lateral de Inspeção (Transaction Drawer)**.

4. Analise a **Timeline do Worker**: visualize os passos `CREATED` → `SQS_ENQUEUED` → `SETTLEMENT_STARTED` → `SETTLED` com a latência calculada em milissegundos.
5. Clique em **"Exportar Livro-Razão (CSV)"** para baixar a planilha auditável formatada com todos os dados.

TELA 3: Laboratório de Carga & Concorrência (#stress) HIGH-THROUGHPUT

Objetivo: Testar o pool de threads e capacidade de processamento em lote sob alta concorrência.

1. Acesse a aba **"Stress Lab"**.
2. Selecione a quantidade de transações para disparo simultâneo (5, 10 ou 20 transações).
3. Clique no botão **"Disparar Lote de Concorrência"**.
4. **Resultado Esperado:** O motor Spring Boot processa as requisições em threads assíncronas paralelas. A interface exibe a taxa de Throughput (TPS - Transações por Segundo), a latência total em milissegundos e a lista de cobranças emitidas no lote.

TELA 4: Auditoria de Webhooks & Verificador Criptográfico (#webhooks) HMAC-SHA256 & REPLAY

Objetivo: Inspecionar notificações assinadas, validar o hash criptográfico e simular tentativas de fraude.

1. Acesse a aba **"Webhooks & HMAC"**.

2.Selecione qualquer notificação enviada na lista lateral.

3. Inspecione o cabeçalho `X-Signature-SHA256` (hash hexadecimal de 64 caracteres) e o payload JSON transmitido.

4. Clique em **"Replay Webhook"**: o motor gera uma nova tentativa de entrega com timestamp e assinatura atualizados.

5. No painel inferior **"Laboratório de Verificação Criptográfica"**, clique em **"Verificar com API Backend"** → Retorno: Assinatura VÁLIDA (Autêntica).

6. Clique no botão **"Simular Fraude / Adulteração"** (que altera o valor da cobrança no JSON) e clique novamente em verificar → Retorno imediato: Assinatura INVÁLIDA (Adulterada), demonstrando a mitigação de fraudes.

TELA 5: Roteiro Interativo com 6 Testes Automatizados em 1-Clique (#tutorial) TESTE GUIADO

Objetivo: Executar todos os testes da homologação através de botões diretos sem precisar digitar nada.

1. Acesse a aba **"Roteiro & Testes"**.

2.Execute os botões sequencialmente:

- **Teste 1:** "Executar Emissão Happy Path (CPF Válido)"
- **Teste 2:** "Testar Rejeição de CPF Inválido (Módulo 11)"
- **Teste 3:** "Emitir Cobrança com CNPJ Empresarial"
- **Teste 4:** "Emitir e Cancelar Imediatamente (Rollback)"
- **Teste 5:** "Disparar Carga Concorrente de 5 Transações"
- **Teste 6:** "Inspecionar Auditoria & HMAC de Webhooks"

3.Todos os testes exibem confirmação com toasts em tempo real e redirecionam para a auditoria dos dados.

4. Guia Passo a Passo: Testes via Terminal & API REST (cURL)

Todos os endpoints podem ser homologados diretamente via terminal utilizando `curl` e `jq` contra a API em nuvem (ou `http://localhost:8080` no ambiente local).

4.1. Cenário 1: Emissão Happy Path (CPF Válido)

```
curl -s -X POST https://pix-flow-backend.onrender.com/api/v1/charges \

  -H "Content-Type: application/json" \

  -d '{

    "pixKey": "52998224725",

    "pixKeyType": "CPF",

    "amount": 150.00,

    "description": "Liquidação em Nuvem",

    "webhookUrl": "https://webhook.site/audit-log"

  }' | jq .
```

Resposta Esperada (HTTP 201 Created):

```
{

  "id": "e4c83a12-8610-41bf-a849-0129bb8e2efb",

  "endToEndId": "E00000000202610111200ABCD12345678",

  "pixKey": "52998224725",

  "pixKeyType": "CPF",

  "amount": 150.00,

  "status": "PENDING",

  "qrCodePayload": "00020126580014br.gov.bcb.pix0136e4c83a12-8610-41bf-a849-
0129bb8e2efb5204000053039865406150.005802BR5913PIXFLOW ENGINE6009SAO
PAULO62070503***6304E8A1",
```

```
  "createdAt": "2026-10-11T12:00:00Z"

}
```

4.2. Cenário 2: Validação Módulo 11 (Rejeição de CPF Inválido)

```
curl -s -i -X POST https://pix-flow-backend.onrender.com/api/v1/charges \

  -H "Content-Type: application/json" \

  -d '{

    "pixKey": "12345678911",

    "pixKeyType": "CPF",

    "amount": 50.00

  }'
```

Resposta Esperada (HTTP 422 Unprocessable Entity):

```
HTTP/2 422

content-type: application/json



{

  "status": 422,

  "error": "UNPROCESSABLE_ENTITY",

  "message": "CPF inválido: os dígitos verificadores informados não atendem ao cálculo
oficial do Módulo 11.",

  "timestamp": "2026-10-11T12:01:00Z"

}
```

4.3. Cenário 3: Cancelamento de Cobrança PENDING

```
# 1. Cria cobrança
```

```
CHARGE_ID=$(curl -s -X POST https://pix-flow-backend.onrender.com/api/v1/charges \

  -H "Content-Type: application/json" \

  -d '{"pixKey":"admin@pixflow.io","pixKeyType":"EMAIL","amount":35.00}' | jq -r .id)



# 2. Cancela antes da liquidação

curl -s -X POST https://pix-flow-backend.onrender.com/api/v1/charges/$CHARGE_ID/cancel | jq
'{id, status}'
```

Resposta Esperada (HTTP 200 OK): `{ "id": "...", "status": "CANCELED" }`

4.4. Cenário 4: Auditoria da Timeline do Worker

```
curl -s https://pix-flow-backend.onrender.com/api/v1/charges/$CHARGE_ID/timeline | jq .
```

Resposta Esperada (Array Cronológico de Eventos):

```
[

  { "previousStatus": null, "newStatus": "PENDING", "latencyMs": 0, "createdAt": "2026-10-
11T12:00:00Z" },

  { "previousStatus": "PENDING", "newStatus": "PROCESSING", "latencyMs": 850, "createdAt":
"2026-10-11T12:00:01Z" },

  { "previousStatus": "PROCESSING", "newStatus": "SETTLED", "latencyMs": 1320, "createdAt":
"2026-10-11T12:00:02Z" }

]
```

4.5. Cenário 5: Verificação Criptográfica de Assinatura HMAC-SHA256

```
curl -s -X POST https://pix-flow-backend.onrender.com/api/v1/webhooks/verify \

  -H "Content-Type: application/json" \

  -d '{

    "payload": "{\"chargeId\":\"e4c83a12-8610-41bf-a849-0129bb8e2efb\",\"status\":
\"SETTLED\",\"amount\":150.00}",
```

```
    "signature": "ba6c72e41f059c2567dfb642ec30fa8e6a578a1098e945c57b85041ff34e2f81"

  }' | jq .
```

Resposta Esperada: `{ "valid": true, "algorithm": "HmacSHA256" }`

5. Guia de Testes Locais com Docker Compose & LocalStack

Para execução 100% offline no Fedora 44 (ou qualquer distribuição Linux), o projeto fornece um `docker-compose.yml` que inicializa o Core Backend, o banco PostgreSQL 16 e o emulador de nuvem AWS **LocalStack 3.7** em um único comando:

```
# 1. Subir toda a infraestrutura local

docker compose up -d --build



# 2. Verificar saúde dos serviços

docker compose ps



# 3. Validar criação das filas SQS no LocalStack

docker exec -it pixflow-localstack awslocal sqs list-queues



# 4. Inspecionar mensagens em tempo real na fila

docker exec -it pixflow-localstack awslocal sqs receive-message \

  --queue-url http://localhost:4566/000000000000/pix-settlement-queue



# 5. Acessar banco PostgreSQL local

docker exec -it pixflow-postgres psql -U pixflow -d pixflow -c "SELECT id, pix_key, status,
amount FROM charges LIMIT 5;"
```

6. Suíte de Testes Automatizados no Backend (141 Testes & 100% JaCoCo)

A esteira de integração contínua do GitHub Actions (`.github/workflows/ci-cd.yml`) impõe a execução mandatória da suíte de testes com JUnit 5, Mockito e MockMvc.

Execução Local no Fedora 44

```
cd backend

chmod +x mvnw

./mvnw clean test jacoco:report
```

Resultados Verificados da Cobertura de Código

Métrica JaCoCo	Total no Projeto	Coberto pelos Testes	Percentual Atingido
Instruções de Bytecode (INSTRUCTION)	3.427	3.427	**100.00%**
Ramificações Lógicas (BRANCH)	173	173	**100.00%**
Linhas de Código (LINE)	782	782	**100.00%**
Métodos (METHOD)	168	168	**100.00%**
Classes de Domínio & Aplicação	35	35	**100.00%**

7. Troubleshooting & FAQ de Homologação

Sintoma / Dúvida	Causa Raiz Técnica	Solução Operacional
Primeira requisição na Vercel demora ~40 segundos.	*Cold start* da instância gratuita do Render (hiberna após 15 min de inatividade).	Aguarde o aquecimento inicial. O banner amarelo na UI informa o estado de reativação. As requisições seguintes responderão em < 50ms.
Erro 422 ao emitir	A chave informada não	Utilize os atalhos de preenchimento rápido

Sintoma / Dúvida	Causa Raiz Técnica	Solução Operacional
cobrança com CPF/CNPJ.	atende à fórmula matemática do Módulo 11 da Receita Federal.	fornecidos no formulário (ex: `529.982.247-25`).
Mensagem não compensada na fila local.	Fila LocalStack não criada na inicialização.	Execute `awslocal sqs create-queue --queue-name pix-settlement-queue` ou reinicie o compose com `docker compose up -d`.
Erro de CORS ao testar via browser local.	Origin `http://localhost:5173` não autorizado.	O `CorsConfig.java` já autoriza `localhost:5173`, `localhost:3000` e `https://*.vercel.app` dinamicamente com credenciais.

PixFlow: Event-Driven Payment & Settlement Engine

Engenharia de Qualidade, Cloud DevOps & Arquitetura de Pagamentos Instantâneos.

Documentação gerada e validada em 2026 para o portfólio de Matheus Araújo Pereira.